DBMS Practical Examination – SQL Select Query Problem Statements

Instructions

- Write and execute SQL queries for the following questions.
- Use appropriate clauses such as WHERE, GROUP BY, HAVING, ORDER BY, JOINS, and aggregate functions wherever applicable.
- Each problem statement contains 7 questions:
  - Q1–Q5: Simple to Moderate Queries
  - Q6–Q7: Medium to Hard Queries

Problem Statement 1 – Library Management System

Schema

BOOK

(Book_ID, Title, Author, Category, Price)

MEMBER

(Member_ID, Member_Name, City)

ISSUE

(Issue_ID, Book_ID, Member_ID, Issue_Date, Return_Date)

Questions

1. Display all books with price greater than 500.
2. Display member details from Pune city.
3. Show book title and member name for issued books.
4. Count total books in each category.
5. Display members who issued books after ‘2026-01-01’.
6. Display the member who issued maximum books.
7. Display category-wise average price of books having average price greater than 400.

Solutions

```sql
SELECT * FROM BOOK WHERE Price > 500;

SELECT * FROM MEMBER WHERE City='Pune';
```

```sql
SELECT B.Title, M.Member_Name
FROM BOOK B
JOIN ISSUE I ON B.Book_ID=I.Book_ID
JOIN MEMBER M ON M.Member_ID=I.Member_ID;

SELECT Category, COUNT(*)
FROM BOOK
GROUP BY Category;

SELECT DISTINCT M.Member_Name
FROM MEMBER M
JOIN ISSUE I ON M.Member_ID=I.Member_ID
WHERE Issue_Date > '2026-01-01';

SELECT M.Member_Name, COUNT(*) AS Total_Books
FROM MEMBER M
JOIN ISSUE I ON M.Member_ID=I.Member_ID
GROUP BY M.Member_Name
ORDER BY Total_Books DESC
LIMIT 1;

SELECT Category, AVG(Price)
FROM BOOK
GROUP BY Category
HAVING AVG(Price) > 400;
```

Problem Statement 2 – Hospital Management System

Schema

DOCTOR

(Doctor_ID, Doctor_Name, Specialization, Salary)

PATIENT

(Patient_ID, Patient_Name, Age, City)

APPOINTMENT

(Appointment_ID, Doctor_ID, Patient_ID, Appointment_Date)

Questions

1. Display all doctors with salary greater than 80000.

2. Display patient details whose age is above 50.
3. Show doctor name and patient name for appointments.
4. Count total doctors specialization-wise.
5. Display appointments after ‘2026-03-01’.
6. Find the doctor handling maximum appointments.
7. Display specialization-wise average salary greater than 70000.

Solutions

```sql
SELECT * FROM DOCTOR WHERE Salary > 80000;

SELECT * FROM PATIENT WHERE Age > 50;

SELECT D.Doctor_Name, P.Patient_Name
FROM DOCTOR D
JOIN APPOINTMENT A ON D.Doctor_ID=A.Doctor_ID
JOIN PATIENT P ON P.Patient_ID=A.Patient_ID;

SELECT Specialization, COUNT(*)
FROM DOCTOR
GROUP BY Specialization;

SELECT * FROM APPOINTMENT
WHERE Appointment_Date > '2026-03-01';

SELECT D.Doctor_Name, COUNT(*) AS Total_Appointments
FROM DOCTOR D
JOIN APPOINTMENT A ON D.Doctor_ID=A.Doctor_ID
GROUP BY D.Doctor_Name
ORDER BY Total_Appointments DESC
LIMIT 1;

SELECT Specialization, AVG(Salary)
FROM DOCTOR
GROUP BY Specialization
HAVING AVG(Salary) > 70000;
```

Problem Statement 3 – Online Shopping System

Schema

CUSTOMER

(Customer_ID, Customer_Name, City)

PRODUCT

(Product_ID, Product_Name, Category, Price)

ORDERS

(Order_ID, Customer_ID, Product_ID, Quantity, Order_Date)

Questions

1. Display products with price less than 1000.
2. Display customers from Mumbai.
3. Show customer name and product name for all orders.
4. Count products category-wise.
5. Display orders placed in April 2026.
6. Find the customer who placed maximum orders.
7. Display category-wise maximum product price.

Solutions

```sql
SELECT * FROM PRODUCT WHERE Price < 1000;

SELECT * FROM CUSTOMER WHERE City='Mumbai';

SELECT C.Customer_Name, P.Product_Name
FROM CUSTOMER C
JOIN ORDERS O ON C.Customer_ID=O.Customer_ID
JOIN PRODUCT P ON P.Product_ID=O.Product_ID;

SELECT Category, COUNT(*)
FROM PRODUCT
GROUP BY Category;

SELECT * FROM ORDERS
WHERE MONTH(Order_Date)=4 AND YEAR(Order_Date)=2026;

SELECT C.Customer_Name, COUNT(*) AS Total_Orders
FROM CUSTOMER C
JOIN ORDERS O ON C.Customer_ID=O.Customer_ID
GROUP BY C.Customer_Name
ORDER BY Total_Orders DESC
LIMIT 1;

SELECT Category, MAX(Price)
FROM PRODUCT
GROUP BY Category;
```

Problem Statement 4 – College Management System

Schema

STUDENT

(Student_ID, Student_Name, Branch, CGPA)

FACULTY

(Faculty_ID, Faculty_Name, Department)

COURSE

(Course_ID, Course_Name, Faculty_ID)

Questions

1. Display students with CGPA greater than 8.
2. Display faculty from IT department.
3. Show course name with faculty name.
4. Count students branch-wise.
5. Display students sorted by CGPA descending.
6. Find the branch having maximum students.
7. Display average CGPA branch-wise having average CGPA above 7.

Solutions

```sql
SELECT * FROM STUDENT WHERE CGPA > 8;

SELECT * FROM FACULTY WHERE Department='IT';

SELECT C.Course_Name, F.Faculty_Name
FROM COURSE C
JOIN FACULTY F ON C.Faculty_ID=F.Faculty_ID;

SELECT Branch, COUNT(*)
FROM STUDENT
GROUP BY Branch;

SELECT * FROM STUDENT
ORDER BY CGPA DESC;

SELECT Branch, COUNT(*) AS Total_Students
FROM STUDENT
GROUP BY Branch
ORDER BY Total_Students DESC
LIMIT 1;
```

```sql
SELECT Branch, AVG(CGPA)
FROM STUDENT
GROUP BY Branch
HAVING AVG(CGPA) > 7;
```

Problem Statement 5 – Banking System

Schema

CUSTOMER

(Customer_ID, Customer_Name, City)

ACCOUNT

(Account_No, Customer_ID, Account_Type, Balance)

TRANSACTION

(Transaction_ID, Account_No, Amount, Transaction_Type)

Questions

1. Display accounts with balance greater than 50000.
2. Display customers from Nashik.
3. Show customer name with account type.
4. Count accounts type-wise.
5. Display debit transactions only.
6. Find the customer having highest balance.
7. Display account type-wise average balance greater than 30000.

Solutions

```sql
SELECT * FROM ACCOUNT WHERE Balance > 50000;

SELECT * FROM CUSTOMER WHERE City='Nashik';

SELECT C.Customer_Name, A.Account_Type
FROM CUSTOMER C
JOIN ACCOUNT A ON C.Customer_ID=A.Customer_ID;

SELECT Account_Type, COUNT(*)
FROM ACCOUNT
GROUP BY Account_Type;

SELECT * FROM TRANSACTION
```

```sql
WHERE Transaction_Type='Debit';

SELECT C.Customer_Name, MAX(A.Balance)
FROM CUSTOMER C
JOIN ACCOUNT A ON C.Customer_ID=A.Customer_ID;

SELECT Account_Type, AVG(Balance)
FROM ACCOUNT
GROUP BY Account_Type
HAVING AVG(Balance) > 30000;
```

Problem Statement 6 – Employee Payroll System

Schema

EMPLOYEE

(Emp_ID, Emp_Name, Department, Salary)

PROJECT

(Project_ID, Project_Name, Budget)

WORKS_ON

(Emp_ID, Project_ID, Hours)

Questions

1. Display employees with salary above 60000.
2. Display projects with budget greater than 100000.
3. Show employee name with project name.
4. Count employees department-wise.
5. Display employees sorted by salary descending.
6. Find the department with highest average salary.
7. Display employees working on more than one project.

Solutions

```sql
SELECT * FROM EMPLOYEE WHERE Salary > 60000;

SELECT * FROM PROJECT WHERE Budget > 100000;

SELECT E.Emp_Name, P.Project_Name
FROM EMPLOYEE E
JOIN WORKS_ON W ON E.Emp_ID=W.Emp_ID
JOIN PROJECT P ON P.Project_ID=W.Project_ID;
```

```sql
SELECT Department, COUNT(*)
FROM EMPLOYEE
GROUP BY Department;

SELECT * FROM EMPLOYEE
ORDER BY Salary DESC;

SELECT Department, AVG(Salary) AS Avg_Salary
FROM EMPLOYEE
GROUP BY Department
ORDER BY Avg_Salary DESC
LIMIT 1;

SELECT Emp_ID, COUNT(Project_ID)
FROM WORKS_ON
GROUP BY Emp_ID
HAVING COUNT(Project_ID) > 1;
```

Problem Statement 7 – Railway Reservation System

Schema

TRAIN

(Train_ID, Train_Name, Source, Destination)

PASSENGER

(Passenger_ID, Passenger_Name, Age)

RESERVATION

(Reservation_ID, Train_ID, Passenger_ID, Journey_Date)

Questions

1. Display trains starting from Pune.
2. Display passengers whose age is greater than 60.
3. Show passenger name with train name.
4. Count reservations train-wise.
5. Display reservations after ‘2026-02-01’.
6. Find the train with maximum reservations.
7. Display source-wise total trains.

Solutions

```sql
SELECT * FROM TRAIN WHERE Source='Pune';

SELECT * FROM PASSENGER WHERE Age > 60;

SELECT P.Passenger_Name, T.Train_Name
FROM PASSENGER P
JOIN RESERVATION R ON P.Passenger_ID=R.Passenger_ID
JOIN TRAIN T ON T.Train_ID=R.Train_ID;

SELECT Train_ID, COUNT(*)
FROM RESERVATION
GROUP BY Train_ID;

SELECT * FROM RESERVATION
WHERE Journey_Date > '2026-02-01';

SELECT Train_ID, COUNT(*) AS Total_Reservations
FROM RESERVATION
GROUP BY Train_ID
ORDER BY Total_Reservations DESC
LIMIT 1;

SELECT Source, COUNT(*)
FROM TRAIN
GROUP BY Source;
```

Problem Statement 8 – Hotel Management System

Schema

ROOM

(Room_No, Room_Type, Charges)

CUSTOMER

(Customer_ID, Customer_Name, City)

BOOKING

(Booking_ID, Room_No, Customer_ID, CheckIn_Date)

Questions

1. Display rooms with charges greater than 3000.
2. Display customers from Delhi.

3. Show customer name with room type.
4. Count rooms type-wise.
5. Display bookings in May 2026.
6. Find the most booked room type.
7. Display average room charges type-wise.

Solutions

```sql
SELECT * FROM ROOM WHERE Charges > 3000;

SELECT * FROM CUSTOMER WHERE City='Delhi';

SELECT C.Customer_Name, R.Room_Type
FROM CUSTOMER C
JOIN BOOKING B ON C.Customer_ID=B.Customer_ID
JOIN ROOM R ON R.Room_No=B.Room_No;

SELECT Room_Type, COUNT(*)
FROM ROOM
GROUP BY Room_Type;

SELECT * FROM BOOKING
WHERE MONTH(CheckIn_Date)=5;

SELECT R.Room_Type, COUNT(*) AS Total_Bookings
FROM ROOM R
JOIN BOOKING B ON R.Room_No=B.Room_No
GROUP BY R.Room_Type
ORDER BY Total_Bookings DESC
LIMIT 1;

SELECT Room_Type, AVG(Charges)
FROM ROOM
GROUP BY Room_Type;
```

Problem Statement 9 – University Examination System

Schema

STUDENT

(Student_ID, Student_Name, Branch)

SUBJECT

(Subject_ID, Subject_Name, Credits)

RESULT

(Student_ID, Subject_ID, Marks)

Questions

1. Display students from Computer branch.
2. Display subjects with credits greater than 3.
3. Show student name with subject name.
4. Count students branch-wise.
5. Display students scoring above 75 marks.
6. Find the topper student.
7. Display average marks subject-wise greater than 70.

Solutions

```sql
SELECT * FROM STUDENT WHERE Branch='Computer';

SELECT * FROM SUBJECT WHERE Credits > 3;

SELECT S.Student_Name, SU.Subject_Name
FROM STUDENT S
JOIN RESULT R ON S.Student_ID=R.Student_ID
JOIN SUBJECT SU ON SU.Subject_ID=R.Subject_ID;

SELECT Branch, COUNT(*)
FROM STUDENT
GROUP BY Branch;

SELECT * FROM RESULT WHERE Marks > 75;

SELECT Student_ID, SUM(Marks) AS Total
FROM RESULT
GROUP BY Student_ID
ORDER BY Total DESC
LIMIT 1;

SELECT Subject_ID, AVG(Marks)
FROM RESULT
GROUP BY Subject_ID
HAVING AVG(Marks) > 70;
```

Problem Statement 10 – Inventory Management System

Schema

PRODUCT

(Product_ID, Product_Name, Stock, Price)

SUPPLIER

(Supplier_ID, Supplier_Name, City)

SUPPLY

(Product_ID, Supplier_ID, Quantity)

Questions

1. Display products with stock less than 20.
2. Display suppliers from Chennai.
3. Show product name with supplier name.
4. Count products supplier-wise.
5. Display products with price between 500 and 2000.
6. Find supplier supplying maximum quantity.
7. Display average product price greater than 1000.

Solutions

```sql
SELECT * FROM PRODUCT WHERE Stock < 20;

SELECT * FROM SUPPLIER WHERE City='Chennai';

SELECT P.Product_Name, S.Supplier_Name
FROM PRODUCT P
JOIN SUPPLY SU ON P.Product_ID=SU.Product_ID
JOIN SUPPLIER S ON S.Supplier_ID=SU.Supplier_ID;

SELECT Supplier_ID, COUNT(*)
FROM SUPPLY
GROUP BY Supplier_ID;

SELECT * FROM PRODUCT
WHERE Price BETWEEN 500 AND 2000;

SELECT Supplier_ID, SUM(Quantity) AS Total_Qty
FROM SUPPLY
GROUP BY Supplier_ID
ORDER BY Total_Qty DESC
```

```sql
LIMIT 1;

SELECT AVG(Price)
FROM PRODUCT
HAVING AVG(Price) > 1000;
```

Problem Statement 11 – Movie Ticket Booking System

Schema

MOVIE

(Movie_ID, Movie_Name, Genre, Rating)

CUSTOMER

(Customer_ID, Customer_Name, City)

BOOKING

(Booking_ID, Movie_ID, Customer_ID, Seats)

Questions

1. Display movies with rating above 8.
2. Display customers from Hyderabad.
3. Show movie name with customer name.
4. Count movies genre-wise.
5. Display bookings with seats greater than 4.
6. Find movie with maximum bookings.
7. Display average rating genre-wise.

Solutions

```sql
SELECT * FROM MOVIE WHERE Rating > 8;

SELECT * FROM CUSTOMER WHERE City='Hyderabad';

SELECT M.Movie_Name, C.Customer_Name
FROM MOVIE M
JOIN BOOKING B ON M.Movie_ID=B.Movie_ID
JOIN CUSTOMER C ON C.Customer_ID=B.Customer_ID;

SELECT Genre, COUNT(*)
FROM MOVIE
```

```sql
GROUP BY Genre;

SELECT * FROM BOOKING WHERE Seats > 4;

SELECT Movie_ID, COUNT(*) AS Total_Bookings
FROM BOOKING
GROUP BY Movie_ID
ORDER BY Total_Bookings DESC
LIMIT 1;

SELECT Genre, AVG(Rating)
FROM MOVIE
GROUP BY Genre;
```

Problem Statement 12 – Sports Club Management System

Schema

PLAYER

(Player_ID, Player_Name, Sport, Age)

COACH

(Coach_ID, Coach_Name, Experience)

TRAINING

(Player_ID, Coach_ID, Hours)

Questions

1. Display players older than 25.
2. Display coaches with experience greater than 10 years.
3. Show player name with coach name.
4. Count players sport-wise.
5. Display players sorted by age descending.
6. Find coach training maximum players.
7. Display sport-wise average age of players.

Solutions

```sql
SELECT * FROM PLAYER WHERE Age > 25;

SELECT * FROM COACH WHERE Experience > 10;
```

```sql
SELECT P.Player_Name, C.Coach_Name
FROM PLAYER P
JOIN TRAINING T ON P.Player_ID=T.Player_ID
JOIN COACH C ON C.Coach_ID=T.Coach_ID;

SELECT Sport, COUNT(*)
FROM PLAYER
GROUP BY Sport;

SELECT * FROM PLAYER ORDER BY Age DESC;

SELECT Coach_ID, COUNT(Player_ID) AS Total_Players
FROM TRAINING
GROUP BY Coach_ID
ORDER BY Total_Players DESC
LIMIT 1;

SELECT Sport, AVG(Age)
FROM PLAYER
GROUP BY Sport;
```

Problem Statement 13 – Airline Reservation System

Schema

FLIGHT

(Flight_ID, Flight_Name, Source, Destination)

PASSENGER

(Passenger_ID, Passenger_Name, Passport_No)

TICKET

(Ticket_ID, Flight_ID, Passenger_ID, Fare)

Questions

1. Display flights from Mumbai.
2. Display passengers whose fare is greater than 10000.
3. Show passenger name with flight name.
4. Count flights source-wise.
5. Display tickets with fare between 5000 and 15000.
6. Find flight generating maximum revenue.
7. Display average fare flight-wise.

Solutions

```sql
SELECT * FROM FLIGHT WHERE Source='Mumbai';

SELECT * FROM TICKET WHERE Fare > 10000;

SELECT P.Passenger_Name, F.Flight_Name
FROM PASSENGER P
JOIN TICKET T ON P.Passenger_ID=T.Passenger_ID
JOIN FLIGHT F ON F.Flight_ID=T.Flight_ID;

SELECT Source, COUNT(*)
FROM FLIGHT
GROUP BY Source;

SELECT * FROM TICKET
WHERE Fare BETWEEN 5000 AND 15000;

SELECT Flight_ID, SUM(Fare) AS Revenue
FROM TICKET
GROUP BY Flight_ID
ORDER BY Revenue DESC
LIMIT 1;

SELECT Flight_ID, AVG(Fare)
FROM TICKET
GROUP BY Flight_ID;
```

Problem Statement 14 – E-Learning Platform

Schema

STUDENT

(Student_ID, Student_Name, Course)

INSTRUCTOR

(Instructor_ID, Instructor_Name, Expertise)

ENROLLMENT

(Student_ID, Instructor_ID, Completion_Percentage)

Questions

1. Display students enrolled in Java course.
2. Display instructors with expertise in Python.

3. Show student name with instructor name.
4. Count students course-wise.
5. Display students with completion percentage above 80.
6. Find instructor teaching maximum students.
7. Display average completion percentage instructor-wise.

Solutions

```sql
SELECT * FROM STUDENT WHERE Course='Java';

SELECT * FROM INSTRUCTOR WHERE Expertise='Python';

SELECT S.Student_Name, I.Instructor_Name
FROM STUDENT S
JOIN ENROLLMENT E ON S.Student_ID=E.Student_ID
JOIN INSTRUCTOR I ON I.Instructor_ID=E.Instructor_ID;

SELECT Course, COUNT(*)
FROM STUDENT
GROUP BY Course;

SELECT * FROM ENROLLMENT
WHERE Completion_Percentage > 80;

SELECT Instructor_ID, COUNT(Student_ID) AS Total_Students
FROM ENROLLMENT
GROUP BY Instructor_ID
ORDER BY Total_Students DESC
LIMIT 1;

SELECT Instructor_ID, AVG(Completion_Percentage)
FROM ENROLLMENT
GROUP BY Instructor_ID;
```

Problem Statement 15 – Vehicle Service Management System

Schema

VEHICLE

(Vehicle_ID, Owner_Name, Vehicle_Type)

MECHANIC

(Mechanic_ID, Mechanic_Name, Experience)

SERVICE

(Service_ID, Vehicle_ID, Mechanic_ID, Service_Cost)

Questions

1. Display vehicles of type Car.
2. Display mechanics with experience greater than 5 years.
3. Show owner name with mechanic name.
4. Count vehicles type-wise.
5. Display services with cost greater than 5000.
6. Find mechanic handling maximum services.
7. Display average service cost mechanic-wise greater than 3000.

Solutions

```sql
SELECT * FROM VEHICLE WHERE Vehicle_Type='Car';

SELECT * FROM MECHANIC WHERE Experience > 5;

SELECT V.Owner_Name, M.Mechanic_Name
FROM VEHICLE V
JOIN SERVICE S ON V.Vehicle_ID=S.Vehicle_ID
JOIN MECHANIC M ON M.Mechanic_ID=S.Mechanic_ID;

SELECT Vehicle_Type, COUNT(*)
FROM VEHICLE
GROUP BY Vehicle_Type;

SELECT * FROM SERVICE WHERE Service_Cost > 5000;

SELECT Mechanic_ID, COUNT(*) AS Total_Services
FROM SERVICE
GROUP BY Mechanic_ID
ORDER BY Total_Services DESC
LIMIT 1;

SELECT Mechanic_ID, AVG(Service_Cost)
FROM SERVICE
GROUP BY Mechanic_ID
HAVING AVG(Service_Cost) > 3000;
```

Problem Statement 16 – Food Delivery Application

Schema

CUSTOMER

(Customer_ID, Customer_Name, City)

RESTAURANT

(Restaurant_ID, Restaurant_Name, Rating)

ORDERS

(Order_ID, Customer_ID, Restaurant_ID, Amount)

Questions

1. Display restaurants with rating above 4.
2. Display customers from Bangalore.
3. Show customer name with restaurant name.
4. Count total orders restaurant-wise.
5. Display orders with amount greater than 1000.
6. Find restaurant receiving maximum orders.
7. Display average order amount restaurant-wise greater than 500.

Solutions

```sql
SELECT * FROM RESTAURANT WHERE Rating > 4;

SELECT * FROM CUSTOMER WHERE City='Bangalore';

SELECT C.Customer_Name, R.Restaurant_Name
FROM CUSTOMER C
JOIN ORDERS O ON C.Customer_ID=O.Customer_ID
JOIN RESTAURANT R ON R.Restaurant_ID=O.Restaurant_ID;

SELECT Restaurant_ID, COUNT(*)
FROM ORDERS
GROUP BY Restaurant_ID;

SELECT * FROM ORDERS WHERE Amount > 1000;

SELECT Restaurant_ID, COUNT(*) AS Total_Orders
FROM ORDERS
GROUP BY Restaurant_ID
ORDER BY Total_Orders DESC
LIMIT 1;
```

```sql
SELECT Restaurant_ID, AVG(Amount)
FROM ORDERS
GROUP BY Restaurant_ID
HAVING AVG(Amount) > 500;
```

Problem Statement 17 – OTT Streaming Platform

Schema

USER

(User_ID, User_Name, Subscription_Type)

SHOWS

(Show_ID, Show_Name, Genre, Rating)

WATCH_HISTORY

(User_ID, Show_ID, Watch_Time)

Questions

1. Display shows with rating greater than 8.
2. Display users having Premium subscription.
3. Show user name with watched show name.
4. Count shows genre-wise.
5. Display watch history with watch time greater than 2 hours.
6. Find most watched show.
7. Display average show rating genre-wise.

Solutions

```sql
SELECT * FROM SHOWS WHERE Rating > 8;

SELECT * FROM USER WHERE Subscription_Type='Premium';

SELECT U.User_Name, S.Show_Name
FROM USER U
JOIN WATCH_HISTORY W ON U.User_ID=W.User_ID
JOIN SHOWS S ON S.Show_ID=W.Show_ID;

SELECT Genre, COUNT(*)
FROM SHOWS
GROUP BY Genre;

SELECT * FROM WATCH_HISTORY WHERE Watch_Time > 2;
```

```sql
SELECT Show_ID, COUNT(*) AS Total_Views
FROM WATCH_HISTORY
GROUP BY Show_ID
ORDER BY Total_Views DESC
LIMIT 1;

SELECT Genre, AVG(Rating)
FROM SHOWS
GROUP BY Genre;
```

Problem Statement 18 – Smart Parking System

Schema

VEHICLE

(Vehicle_ID, Owner_Name, Vehicle_Type)

PARKING_SLOT

(Slot_ID, Floor_No, Charges)

PARKING_RECORD

(Record_ID, Vehicle_ID, Slot_ID, Hours)

Questions

1. Display parking slots with charges greater than 100.
2. Display vehicles of type Bike.
3. Show owner name with slot number.
4. Count parking slots floor-wise.
5. Display parking records with hours greater than 5.
6. Find slot used maximum times.
7. Display average parking hours slot-wise.

Solutions

```sql
SELECT * FROM PARKING_SLOT WHERE Charges > 100;

SELECT * FROM VEHICLE WHERE Vehicle_Type='Bike';

SELECT V.Owner_Name, P.Slot_ID
FROM VEHICLE V
JOIN PARKING_RECORD R ON V.Vehicle_ID=R.Vehicle_ID
JOIN PARKING_SLOT P ON P.Slot_ID=R.Slot_ID;
```

```sql
SELECT Floor_No, COUNT(*)
FROM PARKING_SLOT
GROUP BY Floor_No;

SELECT * FROM PARKING_RECORD WHERE Hours > 5;

SELECT Slot_ID, COUNT(*) AS Usage_Count
FROM PARKING_RECORD
GROUP BY Slot_ID
ORDER BY Usage_Count DESC
LIMIT 1;

SELECT Slot_ID, AVG(Hours)
FROM PARKING_RECORD
GROUP BY Slot_ID;
```

Problem Statement 19 – Online Examination Portal

Schema

STUDENT

(Student_ID, Student_Name, Course)

EXAM

(Exam_ID, Subject_Name, Total_Marks)

RESULT

(Student_ID, Exam_ID, Marks_Obtained)

Questions

1. Display exams with total marks greater than 50.
2. Display students enrolled in AI course.
3. Show student name with subject name.
4. Count students course-wise.
5. Display students scoring above 80.
6. Find student obtaining highest marks.
7. Display subject-wise average marks greater than 70.

Solutions

```sql
SELECT * FROM EXAM WHERE Total_Marks > 50;
```

```sql
SELECT * FROM STUDENT WHERE Course='AI';

SELECT S.Student_Name, E.Subject_Name
FROM STUDENT S
JOIN RESULT R ON S.Student_ID=R.Student_ID
JOIN EXAM E ON E.Exam_ID=R.Exam_ID;

SELECT Course, COUNT(*)
FROM STUDENT
GROUP BY Course;

SELECT * FROM RESULT WHERE Marks_Obtained > 80;

SELECT Student_ID, SUM(Marks_Obtained) AS Total
FROM RESULT
GROUP BY Student_ID
ORDER BY Total DESC
LIMIT 1;

SELECT Exam_ID, AVG(Marks_Obtained)
FROM RESULT
GROUP BY Exam_ID
HAVING AVG(Marks_Obtained) > 70;
```

Problem Statement 20 – EV Charging Station System

Schema

VEHICLE

(Vehicle_ID, Owner_Name, Battery_Capacity)

STATION

(Station_ID, Station_Name, Location)

CHARGING_SESSION

(Session_ID, Vehicle_ID, Station_ID, Units_Consumed)

Questions

1. Display vehicles with battery capacity greater than 50.
2. Display charging stations located in Pune.
3. Show owner name with station name.

4. Count charging sessions station-wise.
5. Display sessions with units consumed greater than 30.
6. Find station with highest total units consumed.
7. Display average units consumed station-wise.

Solutions

```sql
SELECT * FROM VEHICLE WHERE Battery_Capacity > 50;

SELECT * FROM STATION WHERE Location='Pune';

SELECT V.Owner_Name, S.Station_Name
FROM VEHICLE V
JOIN CHARGING_SESSION C ON V.Vehicle_ID=C.Vehicle_ID
JOIN STATION S ON S.Station_ID=C.Station_ID;

SELECT Station_ID, COUNT(*)
FROM CHARGING_SESSION
GROUP BY Station_ID;

SELECT * FROM CHARGING_SESSION WHERE Units_Consumed > 30;

SELECT Station_ID, SUM(Units_Consumed) AS Total_Units
FROM CHARGING_SESSION
GROUP BY Station_ID
ORDER BY Total_Units DESC
LIMIT 1;

SELECT Station_ID, AVG(Units_Consumed)
FROM CHARGING_SESSION
GROUP BY Station_ID;
```

Problem Statement 21 – Freelancing Marketplace

Schema

FREELANCER

(Freelancer_ID, Freelancer_Name, Skill, Rating)

CLIENT

(Client_ID, Client_Name, Country)

PROJECT

(Project_ID, Freelancer_ID, Client_ID, Budget)

Questions

1. Display freelancers with rating above 4.5.
2. Display clients from USA.
3. Show freelancer name with client name.
4. Count freelancers skill-wise.
5. Display projects with budget greater than 50000.
6. Find freelancer handling maximum projects.
7. Display skill-wise average freelancer rating.

Solutions

```sql
SELECT * FROM FREELANCER WHERE Rating > 4.5;

SELECT * FROM CLIENT WHERE Country='USA';

SELECT F.Freelancer_Name, C.Client_Name
FROM FREELANCER F
JOIN PROJECT P ON F.Freelancer_ID=P.Freelancer_ID
JOIN CLIENT C ON C.Client_ID=P.Client_ID;

SELECT Skill, COUNT(*)
FROM FREELANCER
GROUP BY Skill;

SELECT * FROM PROJECT WHERE Budget > 50000;

SELECT Freelancer_ID, COUNT(*) AS Total_Projects
FROM PROJECT
GROUP BY Freelancer_ID
ORDER BY Total_Projects DESC
LIMIT 1;

SELECT Skill, AVG(Rating)
FROM FREELANCER
GROUP BY Skill;
```

Problem Statement 22 – Digital Wallet System

Schema

USER

(User_ID, User_Name, City)

WALLET

(Wallet_ID, User_ID, Balance)

TRANSACTION

(Transaction_ID, Wallet_ID, Amount, Transaction_Type)

Questions

1. Display wallets with balance greater than 10000.
2. Display users from Mumbai.
3. Show user name with wallet balance.
4. Count transactions type-wise.
5. Display credit transactions only.
6. Find wallet with highest balance.
7. Display average transaction amount type-wise.

Solutions

```sql
SELECT * FROM WALLET WHERE Balance > 10000;

SELECT * FROM USER WHERE City='Mumbai';

SELECT U.User_Name, W.Balance
FROM USER U
JOIN WALLET W ON U.User_ID=W.User_ID;

SELECT Transaction_Type, COUNT(*)
FROM TRANSACTION
GROUP BY Transaction_Type;

SELECT * FROM TRANSACTION WHERE Transaction_Type='Credit';

SELECT Wallet_ID, MAX(Balance)
FROM WALLET;

SELECT Transaction_Type, AVG(Amount)
FROM TRANSACTION
GROUP BY Transaction_Type;
```

Problem Statement 23 – Telemedicine System

Schema

DOCTOR

(Doctor_ID, Doctor_Name, Specialization)

PATIENT

(Patient_ID, Patient_Name, City)

CONSULTATION

(Consultation_ID, Doctor_ID, Patient_ID, Fees)

Questions

1. Display doctors specialized in Cardiology.
2. Display patients from Pune.
3. Show doctor name with patient name.
4. Count consultations doctor-wise.
5. Display consultations with fees greater than 1500.
6. Find doctor handling maximum consultations.
7. Display specialization-wise average consultation fees.

Solutions

```sql
SELECT * FROM DOCTOR WHERE Specialization='Cardiology';

SELECT * FROM PATIENT WHERE City='Pune';

SELECT D.Doctor_Name, P.Patient_Name
FROM DOCTOR D
JOIN CONSULTATION C ON D.Doctor_ID=C.Doctor_ID
JOIN PATIENT P ON P.Patient_ID=C.Patient_ID;

SELECT Doctor_ID, COUNT(*)
FROM CONSULTATION
GROUP BY Doctor_ID;

SELECT * FROM CONSULTATION WHERE Fees > 1500;

SELECT Doctor_ID, COUNT(*) AS Total_Consultations
FROM CONSULTATION
GROUP BY Doctor_ID
ORDER BY Total_Consultations DESC
LIMIT 1;
```

```sql
SELECT Specialization, AVG(Fees)
FROM DOCTOR D
JOIN CONSULTATION C ON D.Doctor_ID=C.Doctor_ID
GROUP BY Specialization;
```

Problem Statement 24 – Ride Sharing Application

Schema

DRIVER

(Driver_ID, Driver_Name, Rating)

RIDER

(Rider_ID, Rider_Name, City)

RIDE

(Ride_ID, Driver_ID, Rider_ID, Fare)

Questions

1. Display drivers with rating above 4.
2. Display riders from Chennai.
3. Show driver name with rider name.
4. Count rides driver-wise.
5. Display rides with fare greater than 500.
6. Find driver earning maximum fare.
7. Display average fare driver-wise.

Solutions

```sql
SELECT * FROM DRIVER WHERE Rating > 4;

SELECT * FROM RIDER WHERE City='Chennai';

SELECT D.Driver_Name, R.Rider_Name
FROM DRIVER D
JOIN RIDE RI ON D.Driver_ID=RI.Driver_ID
JOIN RIDER R ON R.Rider_ID=RI.Rider_ID;

SELECT Driver_ID, COUNT(*)
FROM RIDE
GROUP BY Driver_ID;

SELECT * FROM RIDE WHERE Fare > 500;
```

```sql
SELECT Driver_ID, SUM(Fare) AS Total_Fare
FROM RIDE
GROUP BY Driver_ID
ORDER BY Total_Fare DESC
LIMIT 1;

SELECT Driver_ID, AVG(Fare)
FROM RIDE
GROUP BY Driver_ID;
```

Problem Statement 25 – Smart Gym Management System

Schema

MEMBER

(Member_ID, Member_Name, Plan_Type)

TRAINER

(Trainer_ID, Trainer_Name, Experience)

SESSION

(Session_ID, Member_ID, Trainer_ID, Calories_Burned)

Questions

1. Display members with Premium plan.
2. Display trainers with experience greater than 3 years.
3. Show member name with trainer name.
4. Count members plan-wise.
5. Display sessions with calories burned greater than 500.
6. Find trainer handling maximum sessions.
7. Display average calories burned trainer-wise.

Solutions

```sql
SELECT * FROM MEMBER WHERE Plan_Type='Premium';

SELECT * FROM TRAINER WHERE Experience > 3;

SELECT M.Member_Name, T.Trainer_Name
FROM MEMBER M
```

```sql
JOIN SESSION S ON M.Member_ID=S.Member_ID
JOIN TRAINER T ON T.Trainer_ID=S.Trainer_ID;

SELECT Plan_Type, COUNT(*)
FROM MEMBER
GROUP BY Plan_Type;

SELECT * FROM SESSION WHERE Calories_Burned > 500;

SELECT Trainer_ID, COUNT(*) AS Total_Sessions
FROM SESSION
GROUP BY Trainer_ID
ORDER BY Total_Sessions DESC
LIMIT 1;

SELECT Trainer_ID, AVG(Calories_Burned)
FROM SESSION
GROUP BY Trainer_ID;
```

MongoDB Practical Problem Statements

MongoDB Problem Statement 1 – Library Management System

Collection: books

```json
{
  "book_id": 101,
  "title": "Database System",
  "author": "Korth",
  "category": "Education",
  "price": 650,
  "copies": 5
}
```

Queries

1. Display all books.
2. Display books with price greater than 500.
3. Display books belonging to Education category.
4. Display only title and author fields.
5. Count total books.
6. Update copies of a specific book.
7. Delete books with copies less than 2.

Solutions

```
db.books.find();

db.books.find({price: {$gt: 500}});

db.books.find({category: "Education"});

db.books.find({}, {title:1, author:1, _id:0});

db.books.countDocuments();

db.books.updateOne({book_id:101}, {$set:{copies:10}});

db.books.deleteMany({copies: {$lt:2}});
```

MongoDB Problem Statement 2 – Hospital Management System

Collection: patients

```
{
  "patient_id": 201,
  "patient_name": "Rahul Sharma",
  "age": 45,
  "disease": "Diabetes",
  "city": "Pune"
}
```

Queries

1. Display all patients.
2. Display patients with age greater than 40.
3. Display patients from Pune.
4. Display only patient name and disease.
5. Count total patients.
6. Update disease of a patient.
7. Delete patients with age less than 18.

Solutions

```
db.patients.find();

db.patients.find({age: {$gt:40}});

db.patients.find({city:"Pune"});

db.patients.find({}, {patient_name:1, disease:1, _id:0});
```

```
db.patients.countDocuments();

db.patients.updateOne({patient_id:201}, {$set:{disease:"Cancer"}});

db.patients.deleteMany({age: {$lt:18}});
```

MongoDB Problem Statement 3 – Online Shopping System

Collection: products

```
{
  "product_id": 301,
  "product_name": "Laptop",
  "category": "Electronics",
  "price": 55000,
  "stock": 12
}
```

Queries

1. Display all products.
2. Display products with price greater than 50000.
3. Display products in Electronics category.
4. Display only product name and price.
5. Count total products.
6. Update stock of a product.
7. Delete products with stock equal to 0.

Solutions

```
db.products.find();

db.products.find({price: {$gt:50000}});

db.products.find({category:"Electronics"});

db.products.find({}, {product_name:1, price:1, _id:0});

db.products.countDocuments();

db.products.updateOne({product_id:301}, {$set:{stock:20}});

db.products.deleteMany({stock:0});
```

MongoDB Problem Statement 4 – College Management System

Collection: students

```
{
  "student_id": 401,
  "student_name": "Amit Patil",
  "branch": "IT",
  "cgpa": 8.5,
  "city": "Mumbai"
}
```

Queries

1. Display all students.
2. Display students with CGPA greater than 8.
3. Display students from Mumbai.
4. Display only student name and branch.
5. Count total students.
6. Update CGPA of a student.
7. Delete students with CGPA less than 5.

Solutions

```
db.students.find();

db.students.find({cgpa: {$gt:8}});

db.students.find({city:"Mumbai"});

db.students.find({}, {student_name:1, branch:1, _id:0});

db.students.countDocuments();

db.students.updateOne({student_id:401}, {$set:{cgpa:9.1}});

db.students.deleteMany({cgpa: {$lt:5}});
```

MongoDB Problem Statement 5 – Banking System

Collection: accounts

```
{
  "account_no": 5001,
  "customer_name": "Neha Joshi",
  "account_type": "Savings",
  "balance": 85000,
  "city": "Nashik"
}
```

Queries

1. Display all accounts.
2. Display accounts with balance greater than 50000.
3. Display customers from Nashik.
4. Display only customer name and balance.
5. Count total accounts.
6. Update balance of an account.
7. Delete accounts with balance less than 1000.

Solutions

```
db.accounts.find();

db.accounts.find({balance: {$gt:50000}});

db.accounts.find({city:"Nashik"});

db.accounts.find({}, {customer_name:1, balance:1, _id:0});

db.accounts.countDocuments();

db.accounts.updateOne({account_no:5001}, {$set:{balance:90000}});

db.accounts.deleteMany({balance: {$lt:1000}});
```

MongoDB Problem Statement 6 – Food Delivery Application

Collection: orders

```
{
  "order_id": 601,
  "customer_name": "Rohan",
  "restaurant_name": "Spice Hub",
  "amount": 850,
  "status": "Delivered"
}
```

Queries

1. Display all orders.
2. Display orders with amount greater than 500.
3. Display delivered orders.
4. Display only customer name and amount.
5. Count total orders.
6. Update order status.
7. Delete cancelled orders.

Solutions

```
db.orders.find();

db.orders.find({amount: {$gt:500}});

db.orders.find({status:"Delivered"});

db.orders.find({}, {customer_name:1, amount:1, _id:0});

db.orders.countDocuments();

db.orders.updateOne({order_id:601}, {$set:{status:"Completed"}});

db.orders.deleteMany({status:"Cancelled"});
```

MongoDB Problem Statement 7 – OTT Streaming Platform

Collection: shows

```
{
  "show_id": 701,
  "show_name": "Dark Code",
  "genre": "Thriller",
  "rating": 8.9,
  "language": "English"
}
```

Queries

1. Display all shows.
2. Display shows with rating greater than 8.
3. Display Thriller genre shows.
4. Display only show name and rating.
5. Count total shows.
6. Update language of a show.
7. Delete shows with rating less than 5.

Solutions

```
db.shows.find();

db.shows.find({rating: {$gt:8}});

db.shows.find({genre:"Thriller"});

db.shows.find({}, {show_name:1, rating:1, _id:0});
```

```
db.shows.countDocuments();

db.shows.updateOne({show_id:701}, {$set:{language:"Hindi"}});

db.shows.deleteMany({rating: {$lt:5}});
```

MongoDB Problem Statement 8 – Smart Parking System

Collection: parking

```
{
  "vehicle_no": "MH14AB1234",
  "owner_name": "Kiran",
  "slot_no": 12,
  "hours": 4,
  "charges": 200
}
```

Queries

1. Display all parking records.
2. Display records with charges greater than 100.
3. Display records with parking hours greater than 3.
4. Display only owner name and slot number.
5. Count total parking records.
6. Update parking charges.
7. Delete parking records with hours equal to 0.

Solutions

```
db.parking.find();

db.parking.find({charges: {$gt:100}});

db.parking.find({hours: {$gt:3}});

db.parking.find({}, {owner_name:1, slot_no:1, _id:0});

db.parking.countDocuments();

db.parking.updateOne({slot_no:12}, {$set:{charges:250}});

db.parking.deleteMany({hours:0});
```

MongoDB Problem Statement 9 – Ride Sharing Application

Collection: rides

```
{
  "ride_id": 901,
  "driver_name": "Suresh",
  "rider_name": "Anjali",
  "fare": 450,
  "city": "Pune"
}
```

Queries

1. Display all rides.
2. Display rides with fare greater than 300.
3. Display rides from Pune.
4. Display only driver name and fare.
5. Count total rides.
6. Update ride fare.
7. Delete rides with fare less than 50.

Solutions

```
db.rides.find();

db.rides.find({fare: {$gt:300}});

db.rides.find({city:"Pune"});

db.rides.find({}, {driver_name:1, fare:1, _id:0});

db.rides.countDocuments();

db.rides.updateOne({ride_id:901}, {$set:{fare:500}});

db.rides.deleteMany({fare: {$lt:50}});
```

MongoDB Problem Statement 10 – Smart Gym Management System

Collection: members

```
{
  "member_id": 1001,
  "member_name": "Rahul",
  "plan_type": "Premium",
  "trainer": "Akash",
```

```
  "calories_burned": 650
}
```

Queries

1. Display all gym members.
2. Display members with calories burned greater than 500.
3. Display Premium plan members.
4. Display only member name and trainer.
5. Count total members.
6. Update trainer name.
7. Delete members with calories burned less than 100.

Solutions

```
db.members.find();

db.members.find({calories_burned: {$gt:500}});

db.members.find({plan_type:"Premium"});

db.members.find({}, {member_name:1, trainer:1, _id:0});

db.members.countDocuments();

db.members.updateOne({member_id:1001}, {$set:{trainer:"Rohit"}});

db.members.deleteMany({calories_burned: {$lt:100}});
```